



Laboratorio Linux N° 2

Procesos

De manera sencilla diremos que cada programa que esté en ejecución es un proceso, al loggarse y ejecutar una terminal se inicia un proceso. En realidad, un proceso es una instancia de un programa en ejecución, con su propio espacio de direcciones, pila, datos, punteros, registros y cualquier otra información necesaria para que pueda ser ejecutado.

De la misma manera, el sistema inicia procesos que se denominan daemons (demonios), que son procesos que se ejecutan en el background y proveen servicios. Cada proceso tiene un PID (**P**rocess **I**Dentifier), que es un número único (1 a 32768) que el kernel usa para el seguimiento, control y administración de los procesos.

En Linux se distinguen entre estos tres tipos de procesos:

- ✓ Interactivos: Iniciados y controlados por un Shell. Se pueden ejecutar en primer plano (foreground) o en segundo plano (background)
- ✓ En cola: no están asociados a ninguna terminal, se envían a una cola y esperan ejecutarse
- ✓ Daemons: lanzados al iniciar el sistema y se ejecutan en background mientras sean necesarios

Ps (Program Status)

El comando **ps** es el que permite informar sobre el estado de los procesos. **ps** está principalmente basado en el sistema de archivos /proc, es decir, lee directamente la información de los archivos que se encuentran en este directorio.

Los archivos /proc contienen datos que presentan el estado de los procesos y hebras del sistema, los cuales se modifican de manera constante. Se crea cada vez que se inicia el sistema. Se requiere privilegios del root para examinar el directorio en su totalidad. Sin embargo, los archivos relacionados a procesos son propiedad del usuario con el que ejecuta.

Existe un subdirectorio para cada proceso, nombrado con el ID del mismo. En general son archivos de sólo lectura: se puede leer información de estos archivos, pero no modificarlos directamente ni controlar los procesos a través de ellos. Cuando un proceso termina, su directorio desaparece automáticamente.

Algunos archivos contenidos:

- cmdline: contiene el comando que inició el proceso con sus parámetros.
- cwd: enlace simbólico al directorio actual de trabajo (current work directory)
- environ: variables de ambiente del proceso.

De todas maneras, es un directorio que no existe, ya que el tamaño de sus archivos es 0 (no son ni binarios ni texto); porque en Linux todo es administrado como un archivo, en este caso virtuales: no existen en disco, sino que el sistema operativo los crea en memoria si requieres acceder a ellos. Cada uno de ellos contiene un timestamp porque siempre están siendo actualizados.

/proc/cpuinfo y /proc/interrupts proporcionan información sobre el hardware

/proc/filesystems y /proc/partitions proporcionan información sobre el sistema de archivos.

/proc/sys proporciona información sobre la configuración sobre los parámetros del kernel



Otros:

/proc/acpi: datos sobre Advanced Configuration and Power Interface

/proc/cmdline: información sobre los parámetros del kernel al momento del boot time (arranque)

/proc/loadavg: carga promedio del procesador

/proc/devices: información sobre los dispositivos actuales configurados.

La información provista por el comando ps varía dependiendo de la manera en que se use el comando. A continuación, algunos encabezados y sus significados:

PID	Process ID, número único o de identificación del proceso.
PPID	Parent Process ID, padre del proceso
UID	User ID, usuario propietario del proceso
TTY	Terminal asociada al proceso, si no hay terminal aparece entonces un '?'
TIME	Tiempo de uso de cpu acumulado por el proceso
CMD	El nombre del programa o comando que inició el proceso
RSS	Resident Size, tamaño de la parte residente en memoria en kilobytes
SIZE	Tamaño virtual de la imagen del proceso
NI	Nice, valor nice (prioridad) del proceso, un número positivo significa menos tiempo de procesador y negativo más tiempo (-20 a 19)
PCPU	Porcentaje de cpu utilizado por el proceso
STIME	Starting Time, hora de inicio del proceso
STAT	Status del proceso, puede tomar los siguientes valores: • R runnable, en ejecución, corriendo o ejecutándose • S sleeping, proceso en ejecución pero sin actividad por el momento, o esperando por algún evento para continuar • T sTopped, proceso detenido totalmente, pero puede ser reiniciado • Z zombie, difunto, proceso que por alguna razón no terminó de manera correcta, no debe haber procesos zombies

Ejercitación:

~\$ ps -muestra la lista de procesos en ejecución junto con el ID del proceso, el nombre/número de la terminal (TTY), el tiempo de ejecución (TIME) y el nombre del comando que lanza el proceso (CMD).

Argumentos:

~\$ps -a -todos los procesos en ejecución de todos los usuarios del sistema

~\$ps -u -información adicional sobre el porcentaje de uso de memoria y CPU, código de estado y propietario

Abrir dos terminales:

Terminal 1: \$man ps

Terminal 2: \$ ps

\$ ps -e --permite ver la información de todos los procesos en el sistema

\$ ps -f --agrega columnas UID, PPID, C, STIME.

Muchas veces es necesario buscar un proceso por su nombre, el siguiente comando muestra, por defecto, el PID de cada proceso que coincide con el criterio de búsqueda:

\$ pgrep *opciones criterio*

\$ pgrep bash --qué información obtengo?



\$ pgrep -a bash --qué información agrega?
\$ pgrep -c bash --y al cambiar el argumento?

~\$ top

Procesos que consumen muchos recursos (ordena la lista por uso de CPU). Se actualiza de manera

constante (probar con dos terminales a la vez)

Durante su utilización, podemos incluir los comandos k, M (ordena por uso de memoria), N (ordena

por id), z (los procesos en ejecución se muestran en color), h (para ayuda), q (para finalizar).

Ejercitación:

~\$sleep 30

Ctrl+z

~\$sleep 30 & --envía la ejecución del comando al background

~\$jobs --muestra el estado de las tareas (stopped, done, etc) y su id

~\$fg id --trae a foreground

Secuencia:

~\$sleep 30

Ctrl+z

~\$jobs

~\$bg id

~\$jobs

~\$fg id

Enviando señales (signals) a los procesos

Una señal (signal) es un mecanismo de comunicación entre procesos y el sistema operativo. Permite enviar un mensaje a un proceso para indicarle que debe realizar una acción específica.

Cada señal tiene un número y un nombre asociados, y los procesos pueden responder a estas señales ejecutando la acción correspondiente, como finalizar, detenerse temporalmente, etc.

Señal	Número	Descripción
SIGINT Interrupt	2	Interrumpe un proceso (equivalente a Ctrl + C), permite que el proceso realice una limpieza o manejo de recursos antes de finalizar.
SIGKILL Killed	9	Mata un proceso inmediatamente (no puede ser ignorado), se finaliza la ejecución de manera abrupta. No hay posibilidad de recuperación
SIGCONT	18	Reanuda un proceso detenido
SIGSTOP	19	Detiene un proceso, no puede ser ignorado.

Probar las señales con los siguientes comandos:

\$ sleep 1000 & --obtener el número del proceso

\$ kill -2 PID

\$ jobs -l --es una letra ele